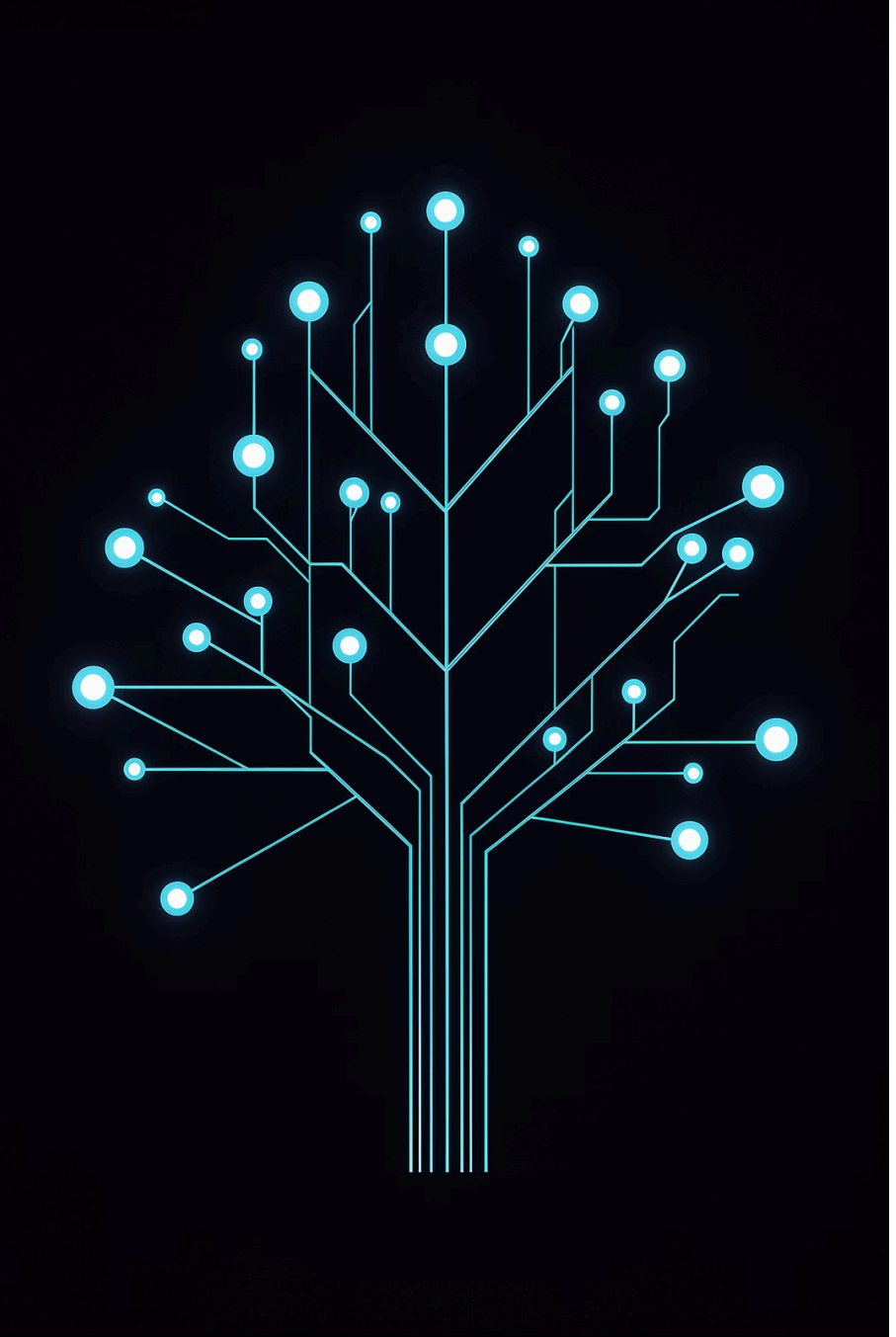


Binary Tree – Part 3

Searching & Aggregation Operations · Balanced Trees & Tree Validation · Tree Views & Basic Structural Problems

ADVANCED DATA STRUCTURES

INTERMEDIATE TO ADVANCED

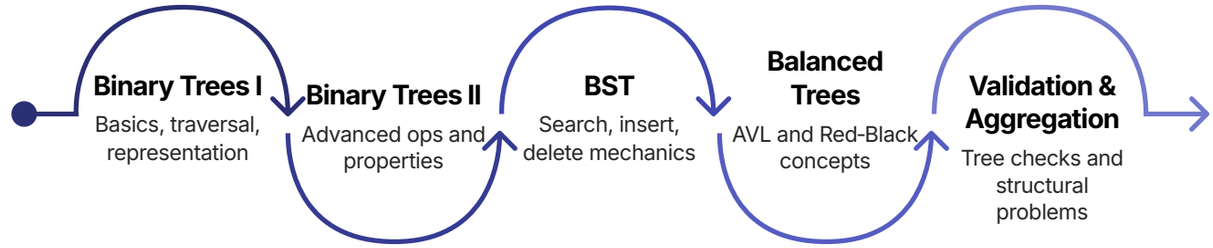
Topic Classification

Topic Name	Binary Tree – Part 3
Category	Advanced Data Structures & Algorithms
Domain	Computer Science → Data Structures → Trees
Topic Type	Algorithmic techniques on binary trees
Difficulty	Intermediate to Advanced
Industry Relevance	Database indexing, compiler optimizations, network routing

Prerequisites

- Binary tree basics (Parts 1 & 2)
- Recursion fundamentals
- Tree traversals (inorder, preorder, postorder)
- Basic understanding of binary search trees (BST)

Recommended Learning Path



What Is It? Core Concepts Defined

Searching in Binary Trees

Locating a node with a given value. O(n) for general trees, but **O(log n)** for BSTs. Compare and go left or right.

Aggregation Operations

Computing a single value from all nodes — sum, max, min, count — using tree traversal. Foundation of database queries.

Balanced Tree

A binary tree where heights of left and right subtrees differ by at most a constant (AVL: ≤1), ensuring O(log n) height.

Tree Validation

Checking whether a binary tree satisfies certain properties — BST property, balance property — to ensure data integrity.

Tree Views

Extracting specific projections — left, right, top, bottom view — as seen from different sides. Used in rendering and debugging.

Structural Problems

Symmetric tree, same tree, subtree check, invert tree — problems that test recursive mirroring and equality.

Complexity at a Glance

Operation	Algorithm	BST Complexity	General Tree
Search	Recursive comparison	O(log n) avg	O(n)
Sum / Aggregate	Any traversal	O(n)	O(n)
Validate BST	Inorder / range check	O(n)	O(n)
Validate balanced	Compute height diff	O(n)	O(n)
Left view	BFS / DFS with level	O(n)	O(n)

Why Does It Exist?

Problems It Solves

- Without BST property, searching takes O(n) — no better than a linked list
- Without balance, BST can degenerate to O(n) height (skewed tree), losing the log advantage
- Tree validation ensures data structure invariants are maintained — critical for library implementations
- Views are needed for printing trees, generating thumbnails, and understanding tree shape

What Happens Without It

- A skewed BST performs as poorly as a linked list
- Inserting into an unbalanced BST without validation leads to unpredictable performance
- No standard way to visualise trees from different angles

Real-World Motivation

Database Index

B-trees (balanced multi-way trees) keep the tree balanced for fast search — prevents O(n) index scans.

Compiler AST

Validating that an abstract syntax tree is well-formed before code generation ensures correctness.

Tree Visualisation

Left/right views help in debugging tree structure in debuggers or GUI tools, avoiding node overlap.

Evolution & History

1

1962

AVL Trees — Adelson-Velsky & Landis. First self-balancing BST, guaranteeing O(log n) height.

2

1972

Red-Black Trees — Bayer. Simpler balancing rules, became the standard for general-purpose maps.

3

1978

Tree Validation Algorithms become standard practice to ensure BST property in implementations.

4

1980s

Aggregation Queries in relational databases — SUM, COUNT on hierarchical data structures.

5

1990s

Tree View Problems appear in coding interviews — left view, right view, boundary traversal.

6

2000s+

Structural Problems (symmetric, subtree, invert) become common in online judges and FAANG interviews.

Today, balanced trees (AVL, Red-Black) are built into standard libraries — C++ `std::map`, Java `TreeMap`. Future trends include persistent balanced trees, concurrent validation, and GPU-accelerated aggregation.

How It Works

BST Search — Recursive

```
function searchBST(node, target):
  if node == null: return null
  if target == node.data: return node
  if target < node.data:
    return searchBST(node.left, target)
  else:
    return searchBST(node.right, target)

Search 60 in tree:
 50 → 60>50 → right(70)
  → 60<70 → left(60) → FOUND
```

Aggregation — Postorder Pattern

```
// Sum of all nodes
function sumTree(node):
  if node == null: return 0
  return node.data
      + sumTree(node.left)
      + sumTree(node.right)

// Maximum value
function maxTree(node):
  if node == null: return -inf
  return max(node.data,
          maxTree(node.left),
          maxTree(node.right))
```

Validate BST — Range Method

```
function isBST(node, min, max):
  if node == null: return true
  if node.data <= min
  or node.data >= max: return false
  return isBST(node.left, min, node.data)
    and isBST(node.right, node.data, max)
```

Validate Balanced (AVL)

```
function isBalanced(node):
  if node == null: return true
  leftH  = height(node.left)
  rightH = height(node.right)
  if abs(leftH - rightH) > 1: return false
  return isBalanced(node.left)
    and isBalanced(node.right)
```

Left View — BFS

```
function leftView(root):
  queue = [root]; result = []
  while queue not empty:
    levelSize = queue.size()
    for i in 0..levelSize-1:
      node = queue.dequeue()
      if i == 0: result.append(node.data)
      if node.left: enqueue(node.left)
      if node.right: enqueue(node.right)
  return result

// Tree: 1→2,3→4,5,_,6
// Left view: 1, 2, 4
```

Symmetric Tree Check

```
function isSymmetric(root):
  return isMirror(root.left, root.right)

function isMirror(t1, t2):
  if t1 == null and t2 == null: return true
  if t1 == null or t2 == null: return false
  return (t1.data == t2.data)
    and isMirror(t1.left, t2.right)
    and isMirror(t1.right, t2.left)
```

Types & Variants

Balanced Tree Comparison

Type	Balance Condition	Height	Use Case
AVL	$\lvert height(L) - height(R) \rvert \leq 1$	~1.44 log n	Frequent lookups, fewer inserts
Red-Black	Color + constraints	≤ 2 log n	General purpose (C++ map)
Weight-balanced	Size(L)/size(R) bounded	O(log n)	Persistent data structures
B-tree (multi-way)	Balanced via node splits	O(log n)	Databases, filesystems

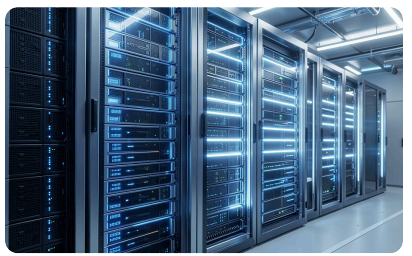
Aggregation Variations

Type	Recursive Formula
Sum	`node.val + sum(L) + sum(R)`
Product	`node.val * prod(L) * prod(R)`
Min	`min(node.val, min(L), min(R))`
Count	`1 + count(L) + count(R)`

Tree Views Comparison

View	Traversal	Output
Left view	BFS or DFS preorder	First node per level
Right view	BFS or DFS right-first	Last node per level
Top view	BFS + horizontal dist	All HDs in order
Bottom view	BFS + HD overwrite	All HDs in order
Boundary	Left + leaves + right	Clockwise perimeter

Real-World Use Cases

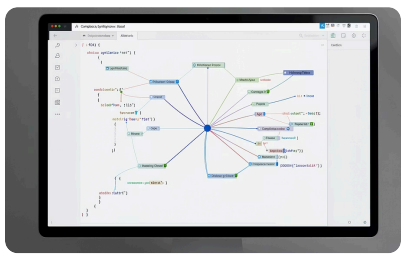


Database Index Validation

Industry: Database Administration

After bulk load or updates, verify that the B-tree index remains balanced; otherwise performance degrades to O(n) scans.

Solution: Traverse the tree, compute height at each node, ensure balance factor within bounds. Detects corruption or implementation bugs.

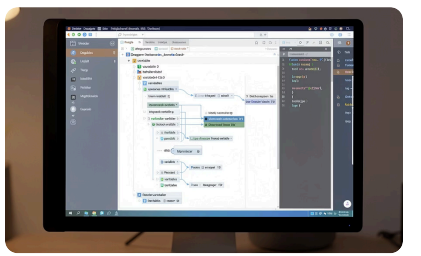


Compiler AST Aggregation

Industry: Programming Language Tooling

Compute total function calls, variable references, or complexity metrics from an abstract syntax tree using recursive postorder aggregation.

Lesson: Use the Visitor pattern to separate aggregation logic from tree structure. One traversal gives multiple metrics.



Tree Rendering in Debuggers

Industry: Developer Tools / IDEs

Render a binary tree as a 2D image; compute left view (first node per level) to determine leftmost x-coordinates and avoid node overlap.

Lesson: Top view is useful for horizontal distance layout in wide trees.

Implementations & Examples

Example 1: BST Search (Iterative)

```
function bstSearch(root, target):
    current = root
    while current != null:
        if target == current.data:
            return current
        else if target < current.data:
            current = current.left
        else:
            current = current.right
    return null
```

Example 2: Validate BST (Range)

```
function isValidBST(root):
    return validate(root, -inf, +inf)

function validate(node, min, max):
    if node == null: return true
    if node.data <= min
    or node.data >= max: return false
    return validate(node.left, min, node.data)
      and validate(node.right, node.data, max)
```

Example 3: Check Balanced (AVL)

```
function isBalanced(root):
    return heightBalanced(root) != -1

function heightBalanced(node):
    if node == null: return 0
    leftH = heightBalanced(node.left)
    if leftH == -1: return -1
    rightH = heightBalanced(node.right)
    if rightH == -1: return -1
    if abs(leftH - rightH) > 1: return -1
    return 1 + max(leftH, rightH)
```

Example 4: Left View (BFS)

```
function leftView(root):
    if root == null: return []
    result = []
    queue = [root]
    while queue not empty:
        levelSize = queue.size()
        for i in 0 to levelSize-1:
            node = queue.dequeue()
            if i == 0:
                result.append(node.data)
            if node.left:
                queue.enqueue(node.left)
            if node.right:
                queue.enqueue(node.right)
    return result
```

Example 5: Symmetric Tree

```
function isSymmetric(root):
    return isMirror(root.left, root.right)

function isMirror(t1, t2):
    if t1 == null and t2 == null:
        return true
    if t1 == null or t2 == null:
        return false
    return (t1.data == t2.data)
      and isMirror(t1.left, t2.right)
      and isMirror(t1.right, t2.left)
```

All five examples follow the same recursive decomposition principle: solve for null base case, then combine results from left and right subtrees.

Hands-On Labs

LAB 1 INTERMEDIATE

Validate a BST and Balanced Tree

Objective: Write functions to check BST property and AVL balance; test on valid and invalid trees.

1. Build a valid BST: [5, 3, 7, 2, 4, 6, 8]
2. Build an invalid BST (e.g., 5→left→3→right→6 where 6 > 5)
3. Run `isValidBST` — should return false
4. Build a balanced and an unbalanced tree; run `isBalanced`
5. Observe that a valid BST can be unbalanced (skewed)

Key Learning: BST property and balance are independent — a valid BST can still be skewed and slow.

LAB 2 ADVANCED

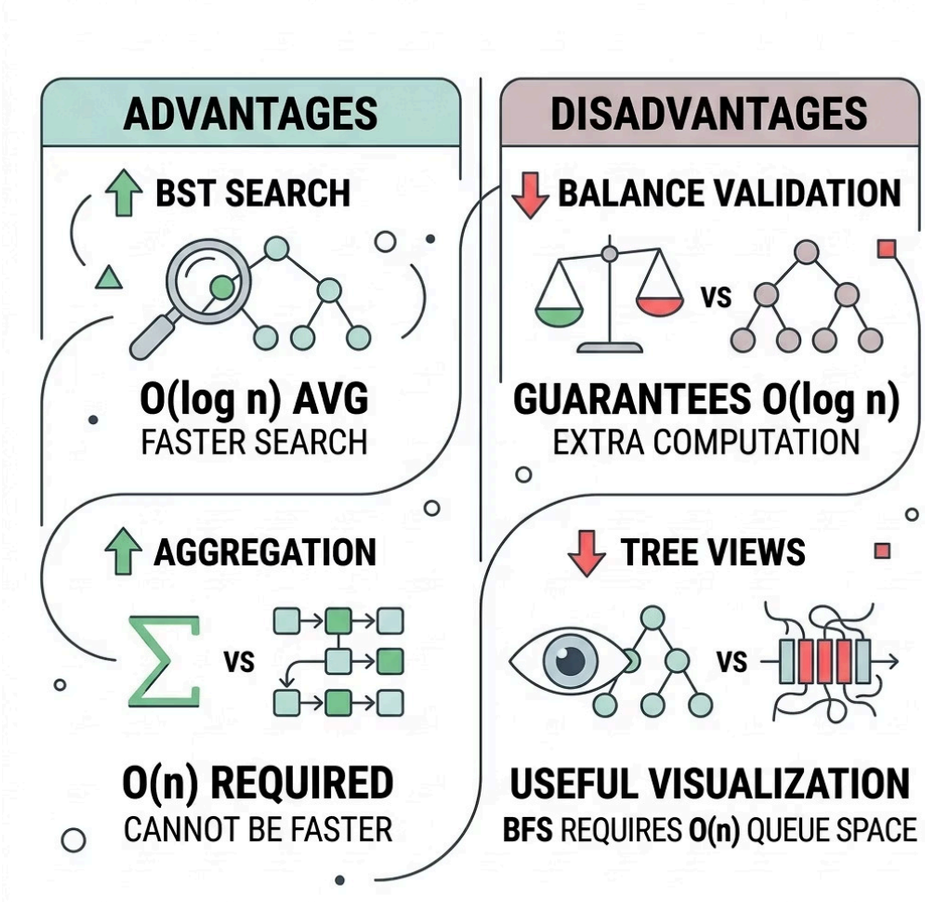
Compute Left View and Top View

Objective: Implement left view and top view; compare outputs on the same tree.

1. Create a binary tree with multiple levels and varying horizontal distances
2. Implement left view (BFS — first node per level)
3. Implement top view using BFS + map of horizontal distance → node value
4. Draw the tree manually and verify outputs match expectations

Key Learning: Left view may include nodes not in top view if they are deeper and not the first horizontally. Level-based vs. horizontal-distance-based views differ.

Advantages & Disadvantages



Summary Table

Feature	Advantage	Disadvantage
BST Search	O(log n) average, fast	O(n) worst case if unbalanced
Aggregation	O(n), necessary for full computation	Cannot be done faster than O(n)
Balance Validation	Ensures O(log n) performance	Requires extra height computation
Tree Views	Useful for visualisation	BFS requires O(n) queue space

⚠ A valid BST is not necessarily balanced. Always validate both properties independently when performance guarantees are required.

Performance Considerations

O(log n)

BST Search

Time complexity on a balanced tree. Degrades to O(n) on skewed trees.

O(n)

Validation & Aggregation

All validation and aggregation operations require visiting every node exactly once.

n/2

Max BFS Queue

BFS queue size can reach n/2 at the last level of a perfect binary tree.

~24

Recursion Depth

For 10^7 balanced nodes, recursion depth is only ~24 — safe for most stacks.

Complexity Table

Operation	Time	Space (recursive)
Search (BST, balanced)	O(log n)	O(log n) / O(1) iterative
Validate BST (range)	O(n)	O(log n) average
Validate balanced	O(n)	O(log n) average
Aggregation (sum, count)	O(n)	O(log n) average
Left / right view	O(n)	O(n) queue

Common Bottlenecks & Optimizations

Recursion Depth

Skewed trees cause stack overflow. Use **iterative implementations** for validation on unknown-depth trees.

Repeated Height Computation

Naive balance check is O(n log n). Fix: use **postorder that returns height** in the same pass.

Map Overhead in Views

Top/bottom view stores horizontal distances. Use array if HD range is known in advance.

Scalability & Reliability

Scaling Strategies by Tree Size

Tree Size	Search	Aggregation	Validation
$< 10^5$	Recursive OK	Recursive OK	Recursive OK
$10^5 – 10^7$	Iterative preferred	Iterative (stack)	Iterative (stack)
$> 10^7$	Distributed / external	Map-reduce	Sample-based

Small (10^4 nodes)

All methods — recursive and iterative — work fine. No special considerations needed.

Large (10^7 nodes, balanced)

Recursion depth ~24, technically safe. But iterative is still safer and avoids stack frame overhead.

Enterprise (10^9 rows)

Database index validation on a billion-row B-tree uses page-level checks, not node-by-node recursion.

Validation functions are read-only and can run concurrently with reads — no locking required for pure validation passes.

Design & Architectural Considerations

Key Architecture Decisions

Recursive vs Iterative Validation

Recursive is simpler to write; iterative is required for deep trees of unknown depth to avoid stack overflow.

Range Limits for BST Validation

Use `long` or sentinel `null` for min/max to handle large values and avoid integer overflow edge cases.

Cache Computed Properties

Store height and size in node metadata to avoid re-traversal on repeated balance checks (AVL approach).

View Algorithm Choice

BFS is straightforward for wide trees; DFS with map may be better for deep trees. Choose based on tree shape.

Design Patterns

Visitor Pattern

Separate validation and aggregation logic from tree structure. Enables multiple operations without modifying nodes.

Decorator Pattern

Add balancing validation to an existing BST without changing its core implementation.

Strategy Pattern

Pluggable validation rules — BST only, balanced only, or both — selected at runtime.

Trade-off: Accuracy (full validation) vs performance (sample validation for large trees). Keep validation functions pure — no side effects.

Security Considerations

Denial of Service

Validation on a maliciously crafted deep tree could cause recursion stack overflow. Always use **iterative validation** for user-provided trees.

Logic Errors

Incorrect validation (e.g., BST check using only local parent-child comparisons) may accept invalid trees, leading to incorrect behaviour downstream.

Integer Overflow

Using `INT_MIN` / `INT_MAX` as sentinels in BST validation can cause false positives. Use `null` optional or `long` with infinity.

Best Practice: For any user-provided or externally sourced tree, always use iterative validation and enforce a maximum tree depth limit before processing.

Best Practices

✅ DO'S

Area	Best Practice
Design	Use postorder aggregation for sum, count, height, and balance — combines bottom-up naturally.
Development	For BST validation, use the range method — O(n), not O(n^2) by checking each node's subtree independently.
Security	Validate tree structure before any recursive processing on user-provided input.
Performance	Cache frequently used aggregations (size, sum) in node metadata to avoid repeated traversals.
Operations	Log validation failures with enough context (node value, reason) for debugging.

❌ DON'TS

Area	Avoid
Design	Don't assume a tree is balanced unless explicitly verified — always check.
Development	Don't use recursion for validation on trees of unknown depth — risk of stack overflow.
Security	Don't skip boundary checks in BST validation — define a clear policy for duplicate keys.
Performance	Don't recompute height multiple times — return it from the same postorder function call.

Common Mistakes & Pitfalls

BEGINNER

Mistake	Why It Happens	Impact	Correct Approach
BST validation checks only parent-child	Local check seems sufficient	Accepts invalid BST	Check entire range (min, max) at every node
Confusing height and depth in balance	Terminology confusion	Wrong balance factor	Height = edges to deepest leaf
Null pointer during tree view	Not checking children before enqueue	Crash / NullPointerException	Always check node != null before enqueuing

INTERMEDIATE

Mistake	Why It Happens	Impact	Correct Approach
Inorder BST validation without tracking previous	Works for unique values	Fails for duplicates	Use range method instead
Forgetting duplicate key policy in BST	Ambiguous definition	Unpredictable behaviour	Define policy: left ≤ or right > explicitly
Using DFS for top view	Simplicity preference	Wrong horizontal order	BFS ensures correct HD priority

ADVANCED / PRODUCTION

Mistake	Why It Happens	Impact	Correct Approach
Recursive validation on extremely deep tree	Assumed tree is balanced	Stack overflow in production	Use iterative stack-based validation
Not resetting min/max sentinel for large values	Using integer limits	False positives in validation	Use `null` optional or `long` with infinity
Memory leak from queue in view functions	Not clearing references (C++)	Gradual memory exhaustion	Use RAII or rely on garbage collection

Comparison with Alternatives

BST Validation Methods

Method	Time	Space	Notes
Inorder (track previous)	O(n)	O(h)	Works for unique keys only
Range (min-max) recursion	O(n)	O(h)	Handles duplicates cleanly
Iterative stack (range)	O(n)	O(h)	Safe for deep trees

Tree Views — BFS vs DFS

View	BFS	DFS (preorder + level)
Left view	Yes, first per level	Yes, track level first occurrence
Right view	Yes, last per level	Yes, right-first preorder
Top view	Natural (level-by-level)	Complex — needs HD + depth tie-breaker
Bottom view	Yes, overwriting	Same as BFS but BFS is easier

Decision Guide

Validate BST?

Duplicates allowed? → Use **range method (min, max)**
No duplicates? → Inorder (prev) also works, but range is safer.

Need Left View?

Memory abundant? → **BFS with queue**
Memory constrained? → **DFS with level tracking**

Need Top View?

Always prefer **BFS with horizontal distance map** — DFS requires complex depth tie-breaking logic.

Balance Validation?

Use the **postorder height-returning function** — avoids O(n log n) repeated height computation.

Learning Summary

Key Takeaways

1 BST Search is O(log n) — but only when balanced

Balance is not guaranteed automatically; it must be validated or enforced via AVL/Red-Black trees.

2 Aggregation uses postorder traversal

Sum, count, max — combine results from children bottom-up. One traversal gives multiple metrics.

3 BST validation requires global range, not local checks

Pass min/max bounds down the recursion — local parent-child checks are insufficient and accept invalid trees.

4 Tree views use BFS for correctness

Left/right views use first/last per level; top/bottom views use horizontal distance tracking with BFS.

Revision Cheat Sheet

Problem	Solution Pattern
BST search	Compare, go left/right recursively
Sum of nodes	`node.val + sum(L) + sum(R)`
Validate BST	range(min, max)
Validate balanced	`abs(h(L)−h(R)) ≤ 1` + children balanced
Left view	BFS, add first node of each level
Symmetric tree	`mirror(left, right)` recursively
Invert tree	Swap left and right, recurse
Same tree	`val equal && same(L,L) && same(R,R)`

Memory Aids

Recommended Next Topics

- **BST** — insert, delete, find, successor/predecessor
- **AVL Trees** — rotations, insertion with balance updates
- **Red-Black Trees** — color invariants, insertion algorithm
- **Segment Trees** — range queries with aggregation
- **Fenwick Tree (BIT)** — prefix sums on arrays
- **Trie (Prefix Tree)** — string storage and search

> "BST validation: **pass the walls (min/max)**"

> "Left view: **first in line (BFS level)**"

> "Balance is **height difference not too steep**"

> "Aggregate **from leaves up (postorder)**"